GWMME — Google Workspace Migration for Microsoft Exchange — Implementation

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

Outcome . . . 4

Approach . . . 4

Why it is low-risk . . . 4

Solution Overview 5

What this solution is . . . 5

Where GWMME fits (and where it does not) . . . 5

How GWMME works (reference model) . . . 6

Naming convention (used throughout) . . . 6

Deployment model . . . 7

Benefits . . . 7

Limitations (scope boundaries) . . . 7

Requirements 8

R1. GWMME migration host — `Migration Host – GWMME – Primary` . . . 8

R2. Source Exchange / M365 access + read identity — `Source Access – Exchange – App Impersonation` . . . 8

R3. Google destination service identity — `Service Identity – Google – Migration` . . . 9

R4. Target Google users provisioned — `Mapping – Users – Master` . . . 9

R5. Source inventory & sizing — for wave design . . . 10

R6. Mail-flow / cutover coexistence plan + rollback — `Rollback Plan – Cutover – Mail` . . . 10

Global Requirements — Google Workspace & Gemini Solutions 10

1. Google Workspace / Cloud Identity tenant . . . 11

2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 11

3. DNS control for the routed domains . . . 11

4. Identity & provisioning source of truth . . . 12

5. A pilot scope (OU / group) and change control . . . 12

6. Security, compliance & governance stakeholders . . . 12

Implementation 12

Phase A — Assess (read-only) . . . 13

Phase B — Design . . . 13

Phase C — Configure . . . 14

Phase D — Pilot . . . 14

Phase E — Validate the pilot . . . 14

Phase F — Expand (wave by wave), then cut over . . . 15

Rollback summary . . . 15

Testing & Validation 16
T1. Prerequisite readiness (Gate C) . . . 16
T2. Mapping integrity (Gate C) . . . 16
T3. Pilot wave completeness — mail (Gate E) . . . 16
T4. Pilot wave completeness — calendar (Gate E) . . . 17
T5. Pilot wave completeness — contacts (Gate E) . . . 17
T6. Error triage completeness (Gate E) . . . 17
T7. Throttling headroom (Gate D/F) . . . 17
T8. Per-wave production completeness (Gate F, each wave) . . . 17
T9. Cutover mail flow (Gate F, cutover) . . . 17
Troubleshooting 18
1. Throttling / API-limit failures during a wave . . . 18
2. Permission / access errors opening source mailboxes . . . 19
3. Destination (Google) authorization failures . . . 19
4. Incomplete data / delta gaps (validation shortfall) . . . 19
5. Missing / broken recurring calendar events or contact fields . . . 20
6. Mapping errors — wrong target or failed rows . . . 20
7. Host / connectivity issues . . . 20
8. Cutover mail-flow failure . . . 21
Best Practices 21
Tool selection & scope . . . 21
Pilot-first, gate everything . . . 22
Batching & throttling . . . 22
Identity, access & security . . . 22
Data quality & validation . . . 22
Traceability & communication . . . 23
Deliverables Summary 23
Documentation . . . 23
Python tools (5) . . . 24
Outcome . . . 24
Appendix A — PowerShell Scripts 25

Executive Summary

Outcome

Exchange and Microsoft 365 mailbox data — **email, calendar, and contacts** — migrated into Gmail on Google Workspace **complete and intact**, using **Google Workspace Migration for Microsoft Exchange (GWMME)**, Google's free mail-focused migration tool. GWMME is installed and configured, source and destination access are granted, mailboxes are mapped, migrations run in **validated, throttle-aware batches**, and mail flow is cut over to Google under a controlled change window with a rehearsed rollback.

Approach

The engagement follows a disciplined, **pilot-first** model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — with explicit pass/fail gates and a documented rollback for every high-impact change:

- **Assess** the source: mailbox inventory, sizes, and special cases; confirm GWMME is the right tool for the data types in scope (it moves mail/calendar/contacts, not files).
- **Design** the waves, the throttling budget, and the mail-flow/DNS cutover, with the rollback written and approved up front.
- **Configure** the migration host, the Microsoft-side read identity, the Google destination service identity, GWMME itself, and the master source→target mapping.
- **Pilot** a small representative wave, then **Validate** mail, calendar, and contacts completeness against the gates before scaling.
- **Expand** wave by wave — each behind its own validation gate — then run final deltas and cut mail flow over to Google.

Five Python tools (Google Workspace Admin SDK / APIs, plus read-only Microsoft-side reconciliation) automate discovery, prerequisite validation, throttle-aware batch planning, mapping/config building, and post-migration completeness reporting — all **read-only or dry-run by default**, with **no credentials or keys in code**.

Why it is low-risk

- **Pilot-first with hard gates** — nothing scales until a representative pilot proves permissions, connections, and completeness.
- **The genuine risk is isolated and controlled** — data movement deletes nothing on the source and is re-runnable; the real production risk is **MX/DNS cutover**, which is gated, scheduled in a change window, and backed by `Rollback Plan – Cutover – Mail`.
- **Throttle-aware batching** avoids the failed, half-finished runs that come from migrating everything at once.

• **Validated every wave** — mail, calendar, and contacts are compared source-to-destination and every error is triaged before the next wave begins.

• **Right-sized tooling** — GWMME needs no migration server farm, keeping the mail move lightweight, repeatable, and auditable.

The result: a mailbox migration where **mail, calendar, and contacts all land**, delivered at a fraction of the cost and risk of an unstructured cutover.

Solution Overview

What this solution is

This solution is a **procedural implementation playbook for Google Workspace Migration for Microsoft Exchange (GWMME)** — the free Google-provided tool that migrates **email, calendar, and contacts** from Microsoft Exchange (on-premises Exchange Server) and **Microsoft 365 / Exchange Online** into Gmail on Google Workspace. It takes a competent Workspace migration engineer from a cold start to a validated, batched cutover: install and configure GWMME on a suitable host, grant the required Exchange/M365 access and Google service identity, build the user-mapping data GWMME consumes, run a pilot batch, validate mail/calendar/contacts, then scale batches with throttling awareness.

GWMME is a **client-side migration utility**: it runs on a Windows host you control, authenticates to the Microsoft source and to the Google destination, and copies mailbox items over. Because it is a tool with its own installer, UI, and versioned behavior, this playbook covers it at the **procedural level** — architecture, prerequisites, access, mapping design, wave sequencing, throttling, and validation. **Deep GWMME console internals, exact dialog labels, licensing, and support are deferred to Google's own GWMME documentation** and are flagged **"verify for your version"** wherever version- or build-dependent behavior matters. Nothing about the tool's internal behavior is invented here.

Where GWMME fits (and where it does not)

Choosing the right migration tool is a design decision, not a default. GWMME is the correct choice for a specific shape of project:

• **GWMME — mail-focused, smaller-to-mid scale.** Migrates **mail, calendar, and contacts** only.

It is a **single-host, client-side** tool: throughput is bounded by that host, the network, and source/destination throttling. It is ideal for **smaller organizations, departmental moves, or the mail/calendar/contacts portion of a larger program** where a lightweight, no-server- infrastructure tool is preferred. It does **not** migrate Drive/OneDrive/SharePoint files, Teams, or chat history.

• **Google Workspace Migrate — larger scale, more data types.** A **server-based** platform (with a management/node architecture) built for **large enterprises** and broader data types (including **Drive/file data**). Choose it when scale, parallelism, or file migration exceed what a single-host tool handles. (Covered by its own solution; its console internals are deferred to Google's docs.)

• **Third-party tools (BitTitan MigrationWiz, CloudM Migrate, etc.)** — used when you need

coexistence features, multi-workload coverage in one pane, or SaaS-hosted orchestration. Integrated at the pattern level; their internals are deferred to those vendors.

> ***Rule of thumb:*** *if the move is* ***mail/calendar/contacts*** *for a manageable user count and you want zero server infrastructure, GWMME fits. If you must move* ***files at enterprise scale****, plan Google Workspace Migrate (or a third-party tool) for that portion and, optionally, GWMME for mail.*

How GWMME works (reference model)

1. **Host.** GWMME is installed on a **Windows migration host** — a dedicated, well-connected machine

(a jump box / VM) reserved for the migration. This host reaches both the Microsoft source and the Google destination.

1. **Source connection.** GWMME connects to the Exchange source. For **Exchange Online / Microsoft

365 **this is over** Exchange Web Services (EWS) **/ the M365 endpoints; for** on-premises Exchange** it connects to the Exchange server's EWS. GWMME reads mailbox items using a source identity that has been granted the right to read the mailboxes in scope (impersonation / application access — see Requirements).

1. **Destination connection.** GWMME authenticates to Google Workspace using a **Google service

identity** (a service account / OAuth credential authorized for the migration scopes) so it can write mail, calendar, and contacts into each target user's Gmail.

1. **Mapping.** GWMME migrates according to a **user-mapping** input (which source mailbox maps to

which Google target user), typically expressed as a **CSV / mapping file** the tool consumes.

1. **Batches (waves).** You migrate in **batches** — sets of mailboxes run together — rather than all

at once, so you can control load, respect throttling, and validate each wave before the next.

1. **Validation.** After each batch you validate that **mail folders, calendar events, and contacts**

arrived complete, triage per-item errors, and re-run deltas as needed.

> ***Verify for your version:*** *the exact GWMME connection dialogs, supported Exchange/EWS versions, the precise mapping-file columns, and the retry/delta behavior are defined by the GWMME build you download. Confirm them against Google's current GWMME documentation before executing.*

Naming convention (used throughout)

Every artifact this playbook creates is named `<Purpose> – <Scope> – <Stage>` and the **same name is reused** in implementation, testing, troubleshooting, and the scripts:

- `Migration Host – GWMME – Primary` — the Windows host running GWMME.
- `Source Access – Exchange – App Impersonation` — the Microsoft-side identity/permission GWMME reads with.
- `Service Identity – Google – Migration` — the Google service account/credential GWMME writes with.
- `Mapping – Users – Master` — the full source→target user map (superset).

- `Migration Wave – Pilot – 01` … `Migration Wave – Finance – 02` — the batch definitions.
- `Validation Report – Pilot – 01` — the per-wave completeness report.
- `Rollback Plan – Cutover – Mail` — the documented back-out for mail-flow/cutover changes.

Deployment model

This solution follows the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — **pilot-first**, with explicit pass/fail gates and a documented rollback for every production change (mail routing, DNS/MX, and cutover are high-impact):

- **Assess** — inventory source mailboxes, sizes, item counts, and special cases (shared mailboxes, large mailboxes, resource calendars); confirm the source is a fit for GWMME.
- **Design** — decide GWMME vs. alternatives for each data type, define waves, the throttling budget, and the coexistence/cutover approach; write the rollback.
- **Configure** — build the host, grant source and destination access, install GWMME, build the master mapping.
- **Pilot** — run `Migration Wave – Pilot – 01` on a small, representative set.
- **Validate** — confirm mail/calendar/contacts completeness against the gates; triage errors.
- **Expand** — scale batches wave by wave, each behind its own gate, until complete; then cut over.

Benefits

- **Right-sized tool, right-sized effort** — GWMME needs no migration server farm; a single host moves mail/calendar/contacts with a documented, repeatable procedure.
- **Complete mailbox data** — mail, calendar, and contacts validated per wave, so nothing is silently dropped.
- **Throttle-aware batching** — waves sized to the source/destination limits avoid the failed, half-finished runs that come from firing everything at once.
- **Low-risk, pilot-first** — every production change has a pass/fail gate and a rollback.
- **Repeatable & auditable** — named artifacts, mapping CSVs, and validation reports make each wave traceable.

Limitations (scope boundaries)

- **Mail, calendar, contacts only.** No Drive/OneDrive/SharePoint files, no Teams, no chat. Plan a separate tool (Google Workspace Migrate or third-party) for those.
- **Single-host throughput.** Scale is bounded by the host, network, and throttling; very large estates may be better served by a server-based platform.

• **Tool internals deferred.** Exact GWMME dialogs, supported versions, and licensing are Google's to define — always confirm against current GWMME documentation for your build.

• **Not a coexistence engine.** GWMME copies data; it does not provide long-running free/busy or cross-platform mail coexistence. Coexistence, if needed, is a separate design.

Requirements

These are the **solution-specific** prerequisites for a GWMME (Google Workspace Migration for Microsoft Exchange) implementation. They build **on** the shared category requirements in `global-requirements-google.md` — the Google Workspace/Cloud Identity tenant (GR-1), source M365/Entra access (GR-2), DNS control for routed domains (GR-3), identity source of truth (GR-4), a pilot scope and change control (GR-5), and security/compliance stakeholders (GR-6). Do not re-establish those here; the items below are what GWMME specifically adds.

Verify each item before starting Section 03 (Implementation).

> ***Verify for your version.*** *GWMME is a versioned Google tool. Its exact supported Exchange/EWS versions, host OS/.NET prerequisites, mapping-file columns, and permission model are defined by the* ***build you download****. Confirm every tool-specific value below against* ***Google's current GWMME documentation*** *before you rely on it.*

R1. GWMME migration host — `Migration Host – GWMME – Primary`

• **Description:** A dedicated **Windows host** (physical or VM) reserved for the migration, meeting GWMME's documented OS/.NET/runtime prerequisites, with strong, stable connectivity to both the Microsoft source and the internet-facing Google endpoints. Sized with adequate CPU/RAM/disk for logs and working data.

• **Why required:** GWMME is a **client-side** tool; it runs on this host and its throughput/stability depend on it. A shared or under-resourced machine causes throttled, failed, or slow runs.

• **How to verify:** Confirm the host meets the GWMME system requirements for your version (OS version, .NET/runtime), can resolve and reach the source EWS endpoint and Google endpoints, and has disk headroom for logs.

• **How to implement if missing:** Provision a clean, patched Windows VM as `Migration Host – GWMME – Primary`; install the GWMME prerequisites Google lists for your version; restrict access to the migration team.

R2. Source Exchange / M365 access + read identity — `Source Access – Exchange – App Impersonation`

• **Description:** An identity on the **Microsoft source** with the rights to **read the mailboxes in scope **— **typically** application impersonation / application access **so one identity can read every in-scope mailbox — plus network reachability to the source** EWS** endpoint (Exchange Online or

on-prem Exchange).

• **Why required:** GWMME must read each source mailbox's mail, calendar, and contacts. Without the correct impersonation/application permission, GWMME can authenticate but cannot open the mailboxes, producing permission failures.

• **How to verify:** Confirm the identity exists, is granted the impersonation/application access scope over the target mailboxes, and that EWS is reachable and enabled for the source. (Exact grant mechanism — RBAC impersonation role vs. app registration with the required application permission — depends on Exchange Online vs. on-prem; **verify for your version/source.**)

• **How to implement if missing:** On the Microsoft side, create/authorize the read identity and grant it impersonation/application access over the in-scope mailboxes under change control; confirm EWS is enabled. Record it as `Source Access – Exchange – App Impersonation`.

R3. Google destination service identity — `Service Identity – Google – Migration`

• **Description:** A **Google service identity** (service account / OAuth credential) authorized to write **mail, calendar, and contacts** into the target Workspace users, with the migration API scopes GWMME requires enabled and domain-wide authorization granted where the tool needs it.

• **Why required:** GWMME writes into each target user's Gmail/calendar/contacts on the Google side and must be authorized to do so across the domain for the users in scope.

• **How to verify:** In the Google Admin console and Google Cloud, confirm the service identity exists, the required APIs are enabled, and the migration scopes are authorized for the tool. (Exact scopes and authorization steps are defined by the GWMME build — **verify for your version.**)

• **How to implement if missing:** Create the service identity, enable the required APIs, and grant the migration scopes per Google's current GWMME setup documentation. **Do not export or commit its key**; store credentials securely. Record it as `Service Identity – Google – Migration`.

R4. Target Google users provisioned — `Mapping – Users – Master`

• **Description:** The **target Workspace users** for every migrating mailbox already **exist and are licensed **in the destination tenant, and a** master mapping** of source mailbox → target Google user is prepared (the input GWMME consumes, typically a CSV).

• **Why required:** GWMME migrates **into an existing target user**; it does not create accounts. A missing/misnamed target, or a wrong mapping row, sends data to the wrong mailbox or fails the row.

• **How to verify:** Reconcile the source mailbox list against provisioned Google users; confirm every in-scope source has a licensed target and exactly one mapping row.

• **How to implement if missing:** Provision/license the missing target users (via your identity

source of truth / provisioning — GR-4), then build/complete `Mapping – Users – Master`. The assessment script in this solution generates the reconciliation and the mapping skeleton.

R5. Source inventory & sizing — for wave design

• **Description:** A **read-only inventory** of the source: mailbox list, per-mailbox **item counts and sizes**, and special cases (shared mailboxes, resource/room calendars, very large or very old mailboxes, archives).

• **Why required:** Wave sizing and the throttling budget depend on **how much** is moving. Large mailboxes and special cases need their own handling; without sizing you cannot plan safe batches or set expectations for run time.

• **How to verify:** Confirm you can enumerate source mailboxes with counts/sizes (via the Microsoft side) and have flagged special cases.

• **How to implement if missing:** Run the discovery/assessment step (Section 03, Assess) to produce the inventory feeding `Mapping – Users – Master` and the wave plan.

R6. Mail-flow / cutover coexistence plan + rollback — `Rollback Plan – Cutover – Mail`

• **Description:** A decided approach for **mail delivery during and after migration** (does mail keep flowing to the source until cutover, and when do **MX/DNS** change to Google?), with a **documented rollback** for the mail-flow and DNS changes.

• **Why required:** Mail routing, MX, and cutover are **high-impact** production changes; a botched cutover means lost or bounced mail. GWMME copies data but does not route live mail — that is a DNS/mail-flow decision that must have a back-out.

• **How to verify:** Confirm the coexistence/cutover decision is documented, the DNS/MX change window and approvers exist (GR-3, GR-5), and `Rollback Plan – Cutover – Mail` is written and approved.

• **How to implement if missing:** Decide the coexistence model, schedule the MX/DNS cutover under change control, and author `Rollback Plan – Cutover – Mail` (revert MX, re-point clients) before any production cutover.

All Microsoft-side and Google-side steps are performed under the change-control process from GR-5. Tool-specific values (versions, scopes, mapping columns, dialogs) are confirmed against Google's current GWMME documentation — never assumed.

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own

Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The* ***Google Admin console*** *(admin.google.com),* ***Google Cloud / Cloud Identity****, and the* ***Microsoft / Entra*** *sides are authored at exact, current click-by-click detail from the vendors' public documentation.* ***Gemini Enterprise*** *capabilities that are in preview or rolling out are* ***cited or flagged "verify in your tenant"*** *— never invented.* ***Third-party migration tools*** *(e.g. for mail/Drive/Chat data movement) and* ***third-party enterprise apps*** *(ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

- **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).
- **Why required:** All configuration, policy, and API work is performed against this tenant.
- **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.
- **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

- **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).
- **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.
- **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.
- **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

- **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.
- **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.
- **How to verify:** Confirm access to the authoritative DNS zone for each domain.
- **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change

control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model; the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the owners before making production changes.

6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance** stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

Implementation

Execute these steps in order. They follow the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — pilot-first, with explicit pass/fail gates and a rollback for every production change. Every artifact is named `<Purpose> – <Scope> – <Stage>` and reused by that exact name in Testing (05), Troubleshooting (06), and the scripts.

> ***Verify for your version.*** *Steps that touch GWMME's own installer, connection dialogs, mapping-file columns, scopes, or supported Exchange/EWS versions are given at the* ***procedural level****. Confirm the exact clicks/columns/values against* ***Google's current GWMME documentation*** *for the build you download before running against production.*

Phase A — Assess (read-only)

A1. Inventory the source. Using read-only access to the Microsoft source (GR-2), enumerate every in-scope mailbox with **item counts and sizes** and flag special cases (shared mailboxes, resource/room calendars, very large mailboxes, archives). Produce `Source Inventory – Exchange – Assess`. *Tool:* `gwmme_discovery.py` reconciles the source list against provisioned Google users and emits the inventory + a mapping skeleton.

A2. Reconcile targets. Confirm every source mailbox has a **provisioned, licensed** Google target user (R4). List any source with no target (must be provisioned first) and any target with no source (out of scope). Output feeds `Mapping – Users – Master`.

A3. Confirm GWMME fit. Verify the move is **mail/calendar/contacts** only and within a scale GWMME handles (Overview). If file data (Drive/OneDrive/SharePoint) is in scope, plan a **separate** tool for that portion now — do not attempt it with GWMME.

> ***Gate A (Assess → Design):*** *complete inventory with sizes; every in-scope source has a licensed target; GWMME confirmed as the right tool for the data types in scope.* ***Fail → stop:*** *provision missing targets or re-scope the tool choice before proceeding.*

Phase B — Design

B1. Define waves. Group mailboxes into batches `Migration Wave – <Scope> – <NN>` sized to the throttling budget (Phase D) — start with a small representative **pilot** wave (`Migration Wave – Pilot – 01`), then business-unit waves. Keep very large mailboxes and special cases in their own smaller waves. *Tool:* `gwmme_batch_planner.py` proposes throttle-aware waves from the inventory (dry-run).

B2. Set the throttling budget. From source/destination throttling guidance (verify for your source), decide concurrent mailboxes per run and per-wave size. Record it with the wave plan.

B3. Decide coexistence & cutover. Decide how mail flows during migration and when **MX/DNS** cut over to Google (R6, GR-3). Author `Rollback Plan – Cutover – Mail` (revert MX, re-point clients) and get it approved under change control (GR-5).

B4. Define pass/fail gates. Write the completeness criteria each wave must meet in Validate (Phase F) — e.g. mail folder/item counts within tolerance, calendar events present, contacts present, errors triaged.

> ***Gate B (Design → Configure):*** *waves defined and sized; throttling budget set; cutover model +* `Rollback Plan – Cutover – Mail` *approved; validation gates written.* ***Fail → stop:*** *finish the design before touching production access.*

Phase C — Configure

C1. Build the host. Stand up `Migration Host – GWMME – Primary` (R1): patched Windows VM meeting GWMME's prerequisites, with reachability to the source EWS and Google endpoints, and disk headroom for logs. Restrict access to the migration team.

C2. Grant source access. On the Microsoft side, create/authorize `Source Access – Exchange – App Impersonation` (R2) — the read identity with impersonation/application access over the in-scope mailboxes — and confirm EWS is enabled and reachable. *(Exact grant mechanism depends on Exchange Online vs. on-prem — verify for your source.)*

C3. Create the Google service identity. Create `Service Identity – Google – Migration` (R3): enable the required APIs and authorize the migration scopes GWMME needs. **Store its credential securely; do not export or commit any key.** *Tool:* `gwmme_prereq_validator.py` checks host reachability, source/destination access, and mapping readiness **read-only** before you install GWMME.

C4. Install & configure GWMME. On the host, install GWMME per Google's current documentation for your version, and configure its **source connection** (to the Exchange source via EWS using `Source Access – Exchange – App Impersonation`) and its **destination connection** (to Google using `Service Identity – Google – Migration`). *(Installer, dialogs, and connection fields — verify for your version.)*

C5. Build the mapping. Complete `Mapping – Users – Master` — the source→target CSV GWMME consumes — from the assessment output. Derive each wave's mapping subset from it. *Tool:* `gwmme_config_helper.py` builds/validates the mapping CSV and derives per-wave subsets.

> ***Gate C (Configure → Pilot):*** *host ready; source and destination access verified by* `gwmme_prereq_validator.py` *(all checks green); GWMME installed and both connections configured;* `Mapping – Users – Master` *validated.* ***Fail → stop:*** *resolve every failed prereq before running any migration.*

Phase D — Pilot

D1. Run the pilot wave. Execute `Migration Wave – Pilot – 01` — a small, representative set (a handful of mailboxes including at least one with a busy calendar and a full contacts list) — through GWMME using the pilot's mapping subset. Keep to the throttling budget (B2).

D2. Capture logs. Retain GWMME's run logs and per-item results for the pilot for triage in Validate/Troubleshooting.

D3. Do not cut over yet. The pilot proves the pipeline; **no MX/DNS change** happens until pilot validation passes.

> ***Gate D (Pilot → Validate):*** *the pilot wave ran to completion (with errors captured, not ignored).* ***Fail → Troubleshooting (06):*** *fix permission/throttling/connection issues and re-run the pilot before validating.*

Phase E — Validate the pilot

E1. Validate completeness. For each pilot mailbox, compare **source vs. destination** for mail (folder/item counts within tolerance), **calendar** (events present, including recurring), and **contacts** (present). Produce `Validation Report – Pilot – 01`. *Tool:* `gwmme_validation_report.py` builds the per-user mail/calendar/contacts completeness report and flags shortfalls.

E2. Triage errors. Classify every error (permission, throttling, unsupported/oversized item, delta gaps) using Troubleshooting (06). Re-run deltas where needed.

E3. Spot-check quality. Manually open a sample of migrated mailboxes: verify folder structure, a recurring meeting, and a contact render correctly in Gmail.

> ***Gate E (Validate → Expand):*** `Validation Report – Pilot – 01` *shows mail/calendar/contacts complete within tolerance and all errors triaged/resolved; spot-checks pass.* ***Fail → stop:*** *do not scale a broken pipeline — fix and re-validate the pilot first.*

Phase F — Expand (wave by wave), then cut over

F1. Run each production wave. Execute `Migration Wave – <Scope> – <NN>` in sequence, each sized to the throttling budget. Do **not** start the next wave until the current one passes its gate.

F2. Validate each wave. Produce `Validation Report – <Scope> – <NN>` with `gwmme_validation_report.py` and triage errors before proceeding.

F3. Run final deltas. Immediately before cutover for a wave, run a **delta** pass to catch mail that arrived at the source after the initial run.

F4. Cut over mail flow. When all waves for a domain are validated, execute the **MX/DNS cutover** (R6, GR-3) under the approved change window so new mail delivers to Google. `Rollback Plan – Cutover – Mail` is on standby.

F5. Verify post-cutover mail flow. Confirm inbound test mail delivers to Google and no mail bounces.

> ***Gate F (per wave, and at cutover):*** *each wave's validation report passes before the next; after cutover, inbound mail delivers to Google.* ***Fail at cutover → rollback:*** *execute* `Rollback Plan – Cutover – Mail` *(revert MX/DNS, re-point clients), restore source mail flow, then diagnose before re-attempting.*

Rollback summary

Change (high-impact)	Rollback
MX/DNS cutover to Google (`Rollback Plan – Cutover – Mail`)	Revert MX/DNS to the source, re-point clients, confirm source mail flow restored.
Source impersonation/app access grant	Remove the grant if migration is aborted; it is read-only, so no source data changes.
Google service identity scopes	Revoke the migration scopes / disable the service identity when the program completes.
A bad wave run	GWMME copies (does not delete source); re-run the wave into the target after fixing the root cause.

*Data movement itself is low-risk to roll back — GWMME reads the source and writes the destination; it does not delete source data. The genuine production risk is **mail-flow/DNS cutover**, which is why it is gated and has an explicit, approved rollback.*

Testing & Validation

Run these validations at the gates defined in Implementation (03). Each has an **expected pass/fail**. Use the named artifacts (`Migration Wave – …`, `Validation Report – …`) consistently. Validation is **read-only** — comparing source and destination — and never modifies mailboxes.

> ***Verify for your version.*** *Where a check depends on GWMME's own logs or supported behavior, confirm the exact log fields/formats against Google's current GWMME documentation for your build.*

T1. Prerequisite readiness (Gate C)

- **Procedure:** Run `gwmme_prereq_validator.py` against the host, source, and destination — it checks

host reachability to source EWS and Google endpoints, that `Source Access – Exchange – App Impersonation` can open a test mailbox, that `Service Identity – Google – Migration` is authorized, and that `Mapping – Users – Master` is well-formed.

- **Pass:** every check reports OK; no missing targets, no unreachable endpoints, no malformed rows.
- **Fail:** any check red → resolve (Requirements 02 / Troubleshooting 06) **before** installing/running

GWMME. Do not proceed on a partial pass.

T2. Mapping integrity (Gate C)

- **Procedure:** Run `gwmme_config_helper.py --validate` on `Mapping – Users – Master`.
- **Pass:** every in-scope source mailbox maps to exactly one **existing, licensed** Google target;

no duplicate source or target rows; wave subsets sum to the master.

- **Fail:** duplicates, orphan rows, or unmatched targets → correct the mapping; re-run.

T3. Pilot wave completeness — mail (Gate E)

- **Procedure:** After `Migration Wave – Pilot – 01`, run `gwmme_validation_report.py` to compare

source vs. destination mail folder/item counts per user; produce `Validation Report – Pilot – 01`.

- **Pass:** destination mail item counts match source **within the agreed tolerance** for every pilot

user; no folder is missing.

- **Fail:** a user is short beyond tolerance or a folder is missing → triage (throttling? oversized/

unsupported items? delta gap?) per Troubleshooting; re-run a delta; re-validate.

T4. Pilot wave completeness — calendar (Gate E)

- **Procedure:** For each pilot user, compare **calendar events** source vs. destination via the validation report; spot-check a **recurring** meeting and an event with attendees.
- **Pass:** event counts match within tolerance; recurring meetings expand correctly; attendees present.
- **Fail:** missing events or broken recurrence → triage and re-run; re-validate.

T5. Pilot wave completeness — contacts (Gate E)

- **Procedure:** Compare **contacts** counts source vs. destination via the validation report; spot-check a contact with multiple fields (phone, address, notes).
- **Pass:** contact counts match within tolerance; sampled contact fields render correctly in Gmail.
- **Fail:** missing contacts or dropped fields → triage and re-run; re-validate.

T6. Error triage completeness (Gate E)

- **Procedure:** Review GWMME's per-item error output for the pilot; classify every error (permission / throttling / unsupported-or-oversized item / delta).
- **Pass:** every error is classified and either resolved or accepted with a documented reason.
- **Fail:** unexplained errors remain → do not pass the gate; investigate.

T7. Throttling headroom (Gate D/F)

- **Procedure:** Run `gwmme_config_helper.py --throttle-check` to review the run against the throttling budget (concurrent mailboxes, wave size/volume estimate) and check observed retry/throttle signals in the GWMME logs.
- **Pass:** the wave completed within the budget without sustained throttling; no runaway retries.
- **Fail:** sustained throttling/retries → reduce concurrency/wave size (re-plan with `gwmme_batch_planner.py`) before scaling.

T8. Per-wave production completeness (Gate F, each wave)

- **Procedure:** Repeat T3–T7 for each `Migration Wave – <Scope> – <NN>`, producing `Validation Report – <Scope> – <NN>`.
- **Pass:** the wave's report is complete within tolerance and errors triaged **before** the next wave starts.
- **Fail:** hold the sequence; fix and re-validate the current wave.

T9. Cutover mail flow (Gate F, cutover)

• **Procedure:** After MX/DNS cutover, send inbound test mail from an external sender to a migrated user and confirm delivery to Google; confirm no bounces.

• **Pass:** inbound mail delivers to Gmail; no NDRs.

• **Fail:** mail bounces or routes to the old source → execute `Rollback Plan – Cutover – Mail` (revert MX/DNS), restore source flow, diagnose DNS/MX before re-attempting.

Validation summary

Test	Gate	Pass signal
T1 Prereq readiness	C	All checks green
T2 Mapping integrity	C	1:1 source→licensed target, no dups
T3 Mail completeness	E	Item counts within tolerance
T4 Calendar completeness	E	Events + recurrence intact
T5 Contacts completeness	E	Contacts + fields intact
T6 Error triage	E	Every error classified/resolved
T7 Throttling headroom	D/F	No sustained throttling
T8 Per-wave completeness	F	Each wave report passes before next
T9 Cutover mail flow	F	Inbound delivers to Google, no bounces

Troubleshooting

Common failure modes for a GWMME (Google Workspace Migration for Microsoft Exchange) migration and how to resolve them. Symptoms map to the named artifacts from Implementation (03) and the gates in Testing (05). GWMME is a versioned tool — where a symptom depends on its logs or behavior, confirm exact messages against **Google's current GWMME documentation** ("verify for your version").

1. Throttling / API-limit failures during a wave

• **Symptoms:** the wave slows to a crawl, GWMME logs show repeated retries or rate/throttle responses from the source or destination, mailboxes finish far behind estimate.

• **Likely causes:** wave too large or concurrency too high for the source (EWS throttling policy) or destination limits; heavy mailboxes running alongside many others.

• **Resolution:**

1. Re-plan smaller waves with `gwmme_batch_planner.py` (lower `--max-per-wave` / `--max-wave-gb`) and

reduce concurrency.

1. Re-estimate with `gwmme_config_helper.py --throttle-check` to confirm the wave fits the budget.
2. Isolate oversized mailboxes into their own `Big-NN` waves.
3. Verify the source EWS throttling policy for the migration identity (verify for your source).

2. Permission / access errors opening source mailboxes

• **Symptoms:** GWMME authenticates but rows fail with access-denied / cannot-open-mailbox; whole wave

fails identically.

• **Likely causes:** `Source Access – Exchange – App Impersonation` lacks impersonation/application

access over the in-scope mailboxes; EWS disabled; wrong grant mechanism for Exchange Online vs. on-prem.

• **Resolution:**

1. Re-run `gwmme_prereq_validator.py` — it fails fast if the source identity can't open a test mailbox.
2. Confirm the impersonation/application grant covers **all** in-scope mailboxes, not a subset.
3. Confirm EWS is enabled/reachable for the source (verify for your version/source).
4. Re-run the wave after the grant propagates.

3. Destination (Google) authorization failures

• **Symptoms:** GWMME connects to the source but fails writing to Google; auth/scope errors on the

destination.

• **Likely causes:** `Service Identity – Google – Migration` missing required APIs/scopes or domain-wide

authorization; wrong credential.

• **Resolution:**

1. Run `gwmme_prereq_validator.py` (Directory API check) to confirm the service identity authenticates.
2. Enable the required APIs and authorize the migration scopes per Google's current GWMME setup docs

(verify for your version).

1. Confirm the credential in use is the intended service identity — **never** an exported key in code.

4. Incomplete data / delta gaps (validation shortfall)

• **Symptoms:** `Validation Report – <Scope> – <NN>` shows destination counts short of source beyond

tolerance for mail, calendar, or contacts.

• **Likely causes:** items arrived at the source after the initial run (no delta); some items skipped

(unsupported/oversized); partial run due to throttling.

• **Resolution:**

1. Triage per user with `gwmme_validation_report.py` (which category is short).
2. Run a **delta** pass for the affected mailboxes to catch newly arrived mail.
3. Review GWMME per-item errors for skipped/unsupported items; handle or document exceptions.
4. Re-validate; only pass the gate when within tolerance.

5. Missing / broken recurring calendar events or contact fields

• **Symptoms:** event counts match but a recurring meeting is missing/mis-expanded, or contacts lack

fields (phone/address/notes).

• **Likely causes:** item-level conversion edge cases; unsupported property.

• **Resolution:**

1. Spot-check a recurring meeting and a multi-field contact by hand (Testing T4/T5).
2. Check GWMME logs for the specific item's conversion warning (verify for your version).
3. Document known unsupported edge cases; re-run the item if it was a transient/throttled failure.

6. Mapping errors — wrong target or failed rows

• **Symptoms:** data lands in the wrong mailbox, or specific rows fail before any transfer.

• **Likely causes:** duplicate/blank rows, target user not provisioned/licensed, wrong column mapping.

• **Resolution:**

1. Run `gwmme_config_helper.py --validate` on the mapping (catches duplicates, blanks, bad targets).
2. Re-run `gwmme_discovery.py` to reconcile against provisioned Google users; provision any missing

targets.

1. Correct `Mapping – Users – Master`, re-derive the wave subset (`--wave`), re-run.

7. Host / connectivity issues

• **Symptoms:** intermittent drops, GWMME can't reach source or Google, runs stall.

• **Likely causes:** under-resourced or unstable `Migration Host – GWMME – Primary`; network/firewall

blocking EWS or Google endpoints; disk full from logs.

- **Resolution:**

1. Run `gwmme_prereq_validator.py` (endpoint reachability checks).
2. Confirm the host meets GWMME's resource prerequisites and has disk headroom for logs.
3. Open required outbound access to the source EWS and Google endpoints (verify for your version).

8. Cutover mail-flow failure

- **Symptoms:** after MX/DNS cutover, inbound mail bounces (NDRs) or still routes to the old source.
- **Likely causes:** MX/DNS misconfiguration, propagation delay, or premature cutover before all waves validated.
- **Resolution:**

1. Confirm all waves for the domain passed validation **before** cutover.
2. Verify MX/SPF/DKIM/DMARC records for the routed domain (GR-3).
3. If mail is bouncing, execute `Rollback Plan – Cutover – Mail` — revert MX/DNS to the source,

re-point clients, confirm source mail flow restored — then diagnose DNS/propagation before re-attempting (Testing T9).

Escalation

For behavior that appears to be a **tool bug or version-specific limitation** (unexpected item handling, undocumented errors, supported-version questions), escalate to **Google's GWMME documentation and support** — this playbook deliberately defers tool internals to the vendor rather than inventing them. Retain the GWMME run logs and the relevant `Validation Report – …` when you escalate.

Best Practices

Durable practices for a reliable GWMME (Google Workspace Migration for Microsoft Exchange) implementation. They complement the ordered steps in Implementation (03) and the gates in Testing (05).

Tool selection & scope

- **Use GWMME for what it is for.** Mail, calendar, and contacts — smaller-to-mid scale, single host, no server infrastructure. If files (Drive/OneDrive/SharePoint) or Teams/chat are in scope, plan a **separate** tool (Google Workspace Migrate or a third-party) for those. Don't force GWMME past its data types.
- **Right-size against scale.** Very large estates strain a single-host tool. If throughput or parallelism needs exceed one host, evaluate a server-based platform before committing to GWMME.
- **Defer tool internals to the vendor.** Confirm supported Exchange/EWS versions, mapping columns,

scopes, and dialogs against **Google's current GWMME documentation** for your build — flag anything version-dependent as "verify for your version" rather than assuming.

Pilot-first, gate everything

- **Never scale a pipeline you haven't validated.** The pilot exists to prove permissions, connections, and completeness. Only pass `Gate E` when mail, calendar, and contacts are all validated.
- **One wave at a time.** Do not start `Migration Wave – <Scope> – <NN+1>` until `<NN>` passes its validation gate and its errors are triaged.
- **Treat mail-flow/DNS cutover as the real risk.** Data movement is largely reversible (GWMME reads source, writes destination, deletes nothing). MX/DNS cutover is not — gate it, schedule it in a change window, and keep `Rollback Plan – Cutover – Mail` on standby.

Batching & throttling

- **Size waves to the throttling budget, not to convenience.** Firing everything at once triggers source/destination throttling and half-finished runs. Start conservative, measure, then increase.
- **Isolate heavy mailboxes.** Very large or item-heavy mailboxes get their own smaller waves so one slow mailbox doesn't stall a batch.
- **Watch retry/throttle signals in the logs.** Sustained retries mean you are over budget — reduce concurrency/wave size and re-plan (`gwmme_batch_planner.py`) rather than pushing harder.
- **Run a delta immediately before cutover.** Mail keeps arriving at the source during migration; a final delta catches it so nothing is lost at the handover.

Identity, access & security

- **Least privilege, both sides.** Grant `Source Access – Exchange – App Impersonation` only over the in-scope mailboxes; grant `Service Identity – Google – Migration` only the migration scopes it needs.
- **Never commit or export credentials.** Keep the Google service credential and any source secret in a secure store, out of the repo, out of logs. The scripts here use OAuth/ADC and never embed keys.
- **Revoke when done.** Remove the source impersonation grant and disable/revoke the Google service identity's migration scopes at program end.
- **Dedicate and lock down the host.** `Migration Host – GWMME – Primary` is migration-only, patched, and restricted to the migration team.

Data quality & validation

- **Validate completeness, don't assume it.** Compare source vs. destination counts for mail, calendar,

and contacts every wave (`gwmme_validation_report.py`); spot-check recurring meetings and multi-field contacts by hand.

- **Classify every error.** Permission, throttling, unsupported/oversized item, delta gap — an

unexplained error is an unvalidated wave.

- **Keep the mapping authoritative.** `Mapping – Users – Master` is the single source of truth; derive

wave subsets from it, never hand-edit rows in isolation.

Traceability & communication

- **Name and reuse artifacts.** `<Purpose> – <Scope> – <Stage>` across host, access, mapping, waves,

and reports makes every run auditable.

- **Retain logs and reports.** Keep GWMME run logs and each `Validation Report – …` for audit and for

diagnosing later waves.

- **Set user expectations.** Communicate cutover timing, what moves (mail/calendar/contacts) and what

does not (files), and where to find migrated data in Gmail.

Deliverables Summary

This solution delivers a complete, pilot-first playbook for implementing **Google Workspace Migration for Microsoft Exchange (GWMME)** — migrating **mail, calendar, and contacts** from Exchange/M365 into Gmail — plus five Python automation tools. Tool internals are deferred to Google's current GWMME documentation; everything here is the **procedure, sequencing, and automation** around it.

Documentation

- **01 — Solution Overview.** What GWMME migrates, where it fits vs. Google Workspace Migrate and

third-party tools, the reference model (host → source EWS → Google service identity → mapping → batches → validation), the naming convention, benefits, and limitations.

- **02 — Requirements.** Solution-specific prerequisites (migration host, source Exchange/M365 access +

read identity, Google destination service identity, provisioned targets + master mapping, source sizing, cutover coexistence + rollback), each with verify/implement, building on the shared Google category requirements.

- **03 — Implementation.** Ordered **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** steps

with named artifacts, explicit pass/fail gates at each phase, MX/DNS cutover, and a rollback table.

• **05 — Testing & Validation.** Nine validations (prereq readiness, mapping integrity, mail/calendar/

contacts completeness, error triage, throttling headroom, per-wave completeness, cutover mail flow), each with expected pass/fail.

• **06 — Troubleshooting.** Common GWMME failure modes — throttling/API limits, permission/access

errors, incomplete/delta items, connection/mapping issues, cutover mail-flow — with resolutions.

• **07 — Best Practices.** Durable practices for tool selection, pilot-first gating, batching/

throttling, identity/security, data quality, and traceability.

• **Executive Summary.** Outcome, approach, and why the migration is low-risk.

Python tools (5)

All tools are **read-only or dry-run by default**, use **OAuth/ADC** (no credentials or keys in code), and standard-library-first (Google client libraries only where the Admin SDK / Workspace APIs are read).

1. `gwmme_discovery.py` — *Assessment / discovery (read-only).* Reconciles the source mailbox

inventory against provisioned Google users via the Directory API and emits `Mapping – Users – Master` plus a gap report (sources without targets, targets without sources).

1. `gwmme_prereq_validator.py` — *Prerequisite / health check (read-only).* Verifies host

reachability to the source EWS and Google endpoints, that the Google service identity authenticates, and that the master mapping is well-formed — the Gate C readiness check before installing GWMME.

1. `gwmme_batch_planner.py` — *Throttle-aware batch planner (dry-run).* Groups mailboxes into a

pilot wave and throttle-budgeted production waves, isolating oversized mailboxes into dedicated waves; writes the wave assignment back to the mapping.

1. `gwmme_validation_report.py` — *Post-migration validation/report (read-only).* Compares source

vs. destination **mail, calendar, and contacts** counts per user (Gmail/Calendar/People read-only APIs), flags shortfalls beyond tolerance, and writes `Validation Report – <Scope> – <NN>`.

1. `gwmme_config_helper.py` — *Scope/config helper (read-only / generate-only).* Validates mapping

integrity, derives per-wave subset CSVs GWMME consumes, and computes a per-wave throttle-budget / run-time estimate.

Outcome

Exchange/M365 mailboxes migrated to Gmail **complete and intact** — GWMME configured, source and destination access granted, mailboxes mapped, batches run and validated, and mail flow cut over to Google under change control with a rehearsed rollback.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

gwmme_batch_planner.py

gwmme_config_helper.py

gwmme_discovery.py

gwmme_prereq_validator.py

gwmme_validation_report.py